

AdamW Optimizer

What makes AdamW better than Adam Optimizer?

The Adam optimizer is popular in deep learning due to its adaptive learning rate and momentum capabilities. However, a variant called AdamW often outperforms the standard Adam optimizer with L2 regularization. L2 regularization also known as weight decay is not as performant as AdamW because it implements weight decay differently. In Adam + L2 regularization the weight decay term gets added directly to the gradient before Adam's adaptive scaling is applied. What this mean is the decay gets distorted by Adam's hyperparameter learning rate adjustments. The regularization effect ends up uneven across weights. In AdamW, the weight decay is applied separately, after the gradient update step, directly to the weights. This makes the regularization behave more predictably and consistently across all parameters, which is why AdamW tends to generalize better.

What effect does Learning Rate have on the results of the optimizer?

When the learning rate is too high the optimizer overshoots the optimal solution resulting in unstable training and lower accuracy. This is observable in Config 2 and 3 (lr=0.01) which produced lower mean validation accuracies and higher standard deviations compared to the default. A learning rate that is too low causes the model to converge too slowly. With a minimum of 20 epochs, the model does not have enough training iterations to reach it's optimal solution. This is evident in Config 4 and Config 5 (lr = 0.0001), which were stable but underperformed compared to the default lr=0.001.

High mean validation accuracy:	The model is performing well.
Low standard deviation :	The model is consistent across all 5 folds, meaning it's not getting lucky on one particular data split.

What effect does weight decay have on the results of the optimizer?

Weight decay is the hyperparameter that controls how aggressively weights are shrunk after each update. Too high a weight decay (e.g. 0.1) causes the model to forget what it learned too quickly, hurting accuracy. Too low a weight decay risks overfitting, where the model memorizes the training data but fails to generalize on unseen data.

The best configuration (Config 1: lr=0.001, weight_decay=0.01) balanced both hyperparameters, achieving the highest mean validation accuracy of **95.70%** with a low standard deviation of **0.0017**, indicating stable and consistent performance across all 5 folds.

What other hyperparameters are in the AdamW optimizer that were not explored?

In this assignment, we only adjusted two hyperparameters learning rate and weight decay. Beta1, Beta2, and Epsilon are some of the other hyperparameters that were kept at their defaults throughout all the runs.

AdamW Function:

```
class torch.optim.AdamW(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0.01, amsgrad=False, *, maximize=False, foreach=None, capturable=False, differentiable=False, fused=None)
```

Betas: coefficients used for computing running averages of gradient and its square. If a tensor is provided, must be 1-element.

Epsilon: Term added to the denominator to improve numerical stability (default: 1e-8).

Maximize: Maximizes (maximize=True) the objective with respect to the params, instead of minimizing (maximize=False). In deep learning, we are minimizing the loss function not maximizing it.

Differentiable: Whether autograd should occur through the optimizer step in training. Otherwise the step() function runs in a torch.no_grad() context. Setting to True can impair performance, so leaving it False doesn't run autograd through this instance. (Default: False)

Capturable: Whether this instance is safe to capture in a graph. Tensors are only capturable when on supported accelerators.

β_1 controls how much the optimizer remembers previous gradients (momentum). Default is 0.9 meaning it retains 90% of the previous gradient direction.

Higher β_1 (e.g. 0.99)	More momentum, smoother updates, slower to change direction. Good for consistent gradients but slow to adapt to sudden changes.
Lower β_1 (e.g. 0.5)	Less momentum, more reactive to current gradient, but noisier and less stable training.

Varying β_1 has a much smaller effect on final accuracy than varying learning rate or weight decay. PyTorch automatically sets β_1 and β_2 to 0.9 and 0.99 by default.

Weight decay has a more direct and interpretable effect on generalization hence the choice of varying weight decay.